Distributed Software Factory — Spec v0.2 (final)

Oct 24, 2026 · @Wobbly

Purpose and constraints

A hobby team pools its own laptops and model subscriptions to build one project from a shared ticket backlog. Agents do the building; humans hold two gates. This is the final consolidated spec: the working draft (v0.1 through the supersession pass) is kept as the record of how each decision was reached, and this document states only what is decided.

Built from ADLC (lifecycle doctrine, gates, `fleet`), a subset of ACFS (box bootstrap, safety hooks, capacity probe), and the best-in-slot tools the supersession pass found. Where a borrowed tool and a first-party component overlap, the borrowed tool wins unless this spec says otherwise.

Non-negotiables

- Cheap: one public GitHub repo, free Actions minutes, self-hosted runners on members' machines for heavy jobs, no paid service, no shared API key.
- Pull, not push: idle nodes claim work. No scheduler, no coordinator process, no machine that must stay on. Every cross-node service is a GitHub Action or a git branch.
- Heterogeneous nodes: any Linux, macOS or WSL2 box with 8 GB+ RAM and one authenticated coding-agent CLI can join.
- Control flow is code, judgment is models (ADLC D0). No model decides a claim, a gate verdict, a merge order, or a release.
- Two human gates only: spec approval (P1) and behavioral acceptance (P6). Everything between is machine-gated.
- Etiquette by mechanism, not instruction: branch protection, claim TTLs, scoped tokens, a kill switch, quota caps. Nothing that matters lives only in `AGENTS.md`.
- The factory must never leave a member unable to use their own account. Quota cap defaults to 40% of the owner's window and the node sleeps through the owner's working day.

Out of scope: Windows outside WSL2, private repos, cross-project sharing, money changing hands, agent-to-agent messaging (ADLC rejects it; conclusions pass through artifacts).

Decisions baked in

- Ticket source of truth is `.adlc/tickets.json`, mirrored to GitHub Issues for display and human interaction. The lock is a git ref, not the Issue (Claim protocol). Beads is dropped to avoid two ledgers; `tk` is the trial replacement if `tickets.json` becomes painful.
- ACFS supplies the bootstrap, `dcg` and the capacity probe only. Agent Mail and NTM are not used.
- Everything a node needs is in the repo. Fresh clone plus `/factory:join` is the whole onboarding.
- The project license is chosen in `PROJECT.md` before the first merge. Node owners DCO-sign agent commits through the commit trailer, and every agent commit carries an `Assisted-by:` trailer so the project's own AI-contribution disclosure is machine-checkable.
- The factory's first project is the factory: every `factory/` change goes through the full lifecycle before the queue opens to the real project.

Architecture

Hub and spoke. The hub is the GitHub repo: Issues, branches, Actions, the merge queue. Every pod reads and writes the hub; no pod talks to another. Fifteen pods: seven are deterministic code, six are agents, two are humans.

Pod	Kind	Owns
Intake + triage	agent, cheap	request → specified routing by risk × blast radius; trivial work bypasses the desk
Contract desk	human, rotating weekly	approves specs (gate P1), writes nothing from a blank page, handles dead letters
Rail author	agent, fresh context	specified → railed: tests, contracts and a tool policy written blind, then frozen
Claim queue	code, on each node	ready → claimed through the git-ref lock
Builder	agent, via `fleet`	claimed → building → gated, inside a per-ticket sandbox

Pod	Kind	Owns
CI gate	code, Actions	deterministic gates rerun; the laptop is not a trust boundary
Prosecutor	agent, cross-adapter	gated → prosecuting: adversarial review, loop until no gating findings
Verifier	agent, fresh context	prosecuting → verified: reproduce or kill each finding; confirmed ones become fix tickets
Merge queue	code, GitHub	verified → queued → integrated; foundation-first order; ejection is a free rebase strike
Preview env	code, heavy node	an address with a TTL per integration branch
Acceptor	human	integrated → accepted (gate P6), or auto when behavior diff is empty and `touches_factory` is false
Release	code	accepted → released: tag, publish, `adlc ticket complete` on `main`
Clock + janitor	code, hourly Action	every TTL, heartbeat, stale and scheduled transition
Budget governor	code, on each node	any active → paused: quota cap, quiet hours, kill switch, wall clock
Summoner	code, Action + webhook	pulls a named human in at the two gates and on halts; everything else is a daily digest

Three state machines run in the hub. Tickets are the only one agents touch; nodes and claims are entirely clock and code.

Ticket transitions and owners

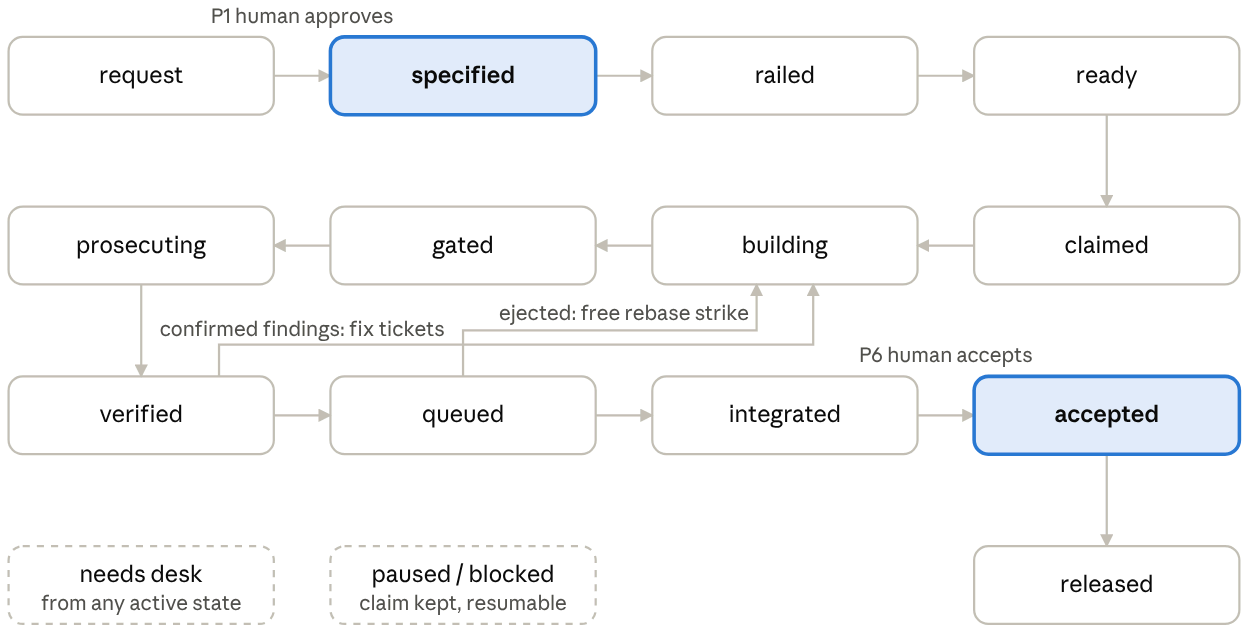
From	To	Owner	Condition
request	specified	intake + desk	P0 routes it; intake drafts; `spec-lint` clean; human approves
specified	railed	rail author	tests, contracts and tool policy authored blind; suite red for the right reasons

From	To	Owner	Condition
railed	ready	CI	`coldstart` gap list empty; `merge-forecast` certifies; else → needs desk
ready	claimed	claim queue	`refs/locks/issue-N` created; back to ready when the lock goes stale (clock)
claimed	building	node	worktree cut, sandbox up, fleet strike started
building	gated	node + CI	rails green, scope clean, tamper check clean; failure → building (strike) or → needs desk
gated	prosecuting	node	`adversarial-review` with a different adapter than the builder where the node has one
prosecuting	verified	verifier	each finding reproduced or killed; confirmed ones spawn fix tickets entering at building; parent waits
verified	queued	merge queue	PR opened with signed gate manifest; `tier:` label sets order
queued	integrated	merge queue + preview env	merged to integration branch, preview deployed
queued	building	merge queue	ejected on conflict; rebase strike, no budget cost
integrated	accepted	human or auto	summoner pings; auto when `behavior-diff` is empty and `touches_factory` is false
accepted	released	release	tag, publish, ticket completed on `main`
any active	paused	budget governor	quota cap, quiet hours, `factory:halt`, wall clock; resumable, claim kept

From	To	Owner	Condition
any active	blocked	CI	a dependency ticket is not yet integrated
any active	needs desk	CI or node	strikes exhausted, flail, `hotfix` label, or a flaky-quarantine event
needs desk	specified or abandoned	desk	rewrite or close

A `hotfix`-labeled ticket skips railed and runs ready → claimed → building against existing rails. A revert is a desk-triggered hotfix whose scope is the reverted PR's file set.

Twelve ticket states, two human gates, two loops back to building



Shaded boxes are the two human gates. Dashed boxes are side states the clock or governor enters and leaves.

ticket state machine · 12 states, 2 human gates, 2 loops back

The only places a ticket re-enters building are merge-queue ejection and verifier-confirmed findings; neither costs the ticket a strike it did not earn.

Node: joining → active → paused → stale (24 h silent) → retired (7 days). Joining and active are written by the node; the rest by the clock. A retired node's token is revoked and its claims released.

Claim: held → released (node or desk) or stale (clock). A claim never changes hands; the lock ref is deleted and re-taken by the next node.

Repository and nodes

One public repo holds both the factory and the thing being built. Tickets that touch `factory/` edit the rules governing them, so they take a stricter path (Trust).

```
PROJECT.md                # charter: what, scope, non-goals, done-means,
license, "this quarter we ship X"
AGENTS.md                 # project-specific agent guidance (untrusted by
design; nothing enforced lives here)
CONTRIBUTING.md           # how a human joins; DCO; AI-disclosure trailer;
provider-terms note
CODEOWNERS                # factory/, .adlc/, .github/, .claude/ and the
quality settings need two humans
factory/
  VERSION                 # nodes refuse to claim while behind
  RULES.md                # house rules, printed by the wizard
  HALT                    # kill switch file; its presence halts every loop
(see Humans)
  skills/join, run, release
  nodes/README.md         # manifest schema only; manifests live on the
nodes branch
  fixtures/               # synthetic data for heavy gates and previews
  ledger/findings.jsonl   # verified prosecution findings, input to distill
  policy/                 # vetto, Ralon, agent-perms, tamperguard,
Vallum/dcg configs
.adlc/tickets.json        # ticket source of truth, written only from main
.github/workflows/        # gates, janitor, mirror, digest, summoner,
distill crons
docs/adr/                 # adrs-managed decision records
```

Branches: `main` is the project. `nodes` is the live node registry (one JSON per node, heartbeat-committed hourly, never merged to `main`). `ticket/<id>/<handle>` are build branches. `flaky-history` holds test history. `refs/locks/*` are claim locks, not branches. Nothing else is long-lived.

Join wizard

A node is any box that has run `/factory:join`, a Claude Code skill in `factory/skills/join/` that probes the machine, installs the toolchain in safe mode, and commits a manifest. It writes nowhere outside the clone and `~/.factory/`.

1. Probe: `nproc`, total and free RAM, free disk, OS, which of `claude`, `codex`, `agy`, `opencode` are installed and authenticated. Timed dry run of the project's full suite sets `heavy_gates`.
2. Install, pinned: `@adlc/cli` and the ADLC plugin; `vetto` (sandbox); `Ralon` (rails path lock); `dcg` (destructive-command hook; `Vallum` instead when the node runs Cursor or Gemini CLI); `tamperguard`; `cux` for account rotation; `adlc init` if `.adlc/` is missing. Refuses to proceed if `--dangerously-skip-permissions` is aliased in the shell. Never vibe mode.
3. Register: fine-grained GitHub token scoped to this repo (contents: write on `ticket/**`, `nodes/**` and `refs/locks/**` only, 90-day expiry); register the box as a self-hosted Actions runner with label `factory-node` in a runner group scoped to this repo.
4. Commit `<handle>.json` to the `nodes` branch and push. That push is the first heartbeat.
5. Print the run-loop command and `RULES.md`.

Manifest

`nodes/<handle>.json`: `handle`, `owner`, `os`, `cpu`, `ram_gb`, `slots`, `heavy_gates`, `adapters` (name → pinned version), `quota_tier`, `quota_cap`, `quiet_hours`, `accounts` (count only, for cux), `factory_version`, `last_seen`, `status`.

- `slots` = floor(free RAM GB / 2) − 1, capped at 8; this is the fleet concurrency the node runs with.
- `heavy_gates` is true if the full suite finished inside the ticket timeout in the wizard's dry run.
- `quota_tier` is self-declared and used only to pick ticket weight; nobody audits it.
- `quota_cap` defaults to 0.4 of the owner's quota window. The governor pauses the node at the cap.
- `quiet_hours` defaults to the owner's working day in their time zone; the node does not claim inside them.

- `adapters` pins CLI versions. A node whose installed CLI drifts from its manifest refuses to claim until the owner re-pins. Three nodes on three versions is three different factories.
- `status` is written by the janitor: `stale` after 24 h without a heartbeat, `retired` after 7 days.

Run loop

One loop per slot. `/factory:run` starts it; it is a shell script, not an agent.

```
loop:
  version_check        # factory/VERSION and adapter pins; behind → pull,
restart, or refuse to claim
  heartbeat            # last_seen into nodes/<handle>.json on the nodes
branch, push (hourly at most)
  kill_switch_check    # factory/HALT present on origin/main → sleep 30m,
continue
  quiet_hours_check    # inside owner's quiet hours → sleep until they end
  quota_check          # cux status; all accounts ≥ quota_cap of window →
sleep until reset
  claim                # git-ref lock protocol; nothing claimable → sleep
10m, continue
  git fetch
  if ticket/<id>/* exists from a lapsed claim: worktree from it, append its
dead-end material to the ticket
  else: git worktree add .worktrees/<id> origin/main
  ralon run --lock .ralon/<id>.lock -- \
  vetto --profile factory/policy/build.toml -- \
    fleet run --tickets <id> --concurrency 1 --no-pr --no-complete \
      --adapter <preferred adapter> --worker-deps <lockfile cache> \
      --max-strikes <budget.strikes> --wall-clock-minutes <budget.minutes> \
      --pre-strike-argv '["cux","status","--json"]' --json
  case reason:
    merged into fleet/run-*     → tamperguard + scope check locally; push
ticket/<id>/<handle>; open PR; comment on Issue
    quota-paused | wall-clock   → keep lock (heartbeat it), push partial
branch, sleep, resume
    strikes-exhausted | flail   → push branch, label Issue needs-desk, delete
lock
    review-unavailable          → delete lock, sleep 30m
    anything else               → delete lock, comment reason, continue
  git worktree remove .worktrees/<id>
```

Two sandbox profiles in `factory/policy/`: `build.toml` denies all network and bounds writes to the worktree and the dependency cache; `agent.toml` allows egress only to the adapter's API domain. fleet's strike runs under `build.toml` for test and build steps and `agent.toml` for the model turns; vetto is fail-closed, so a profile that cannot be applied stops the strike rather than running open.

Transcripts never leave the node. The ledger and PR body carry gate-manifest hashes, reason codes, strikes, adapter names and token counts; never prompts, file contents or session text. `cass` indexes sessions locally for the node owner only; `deja-vu` is the drop-in if `cass` is unavailable on a platform.

Tickets, the desk, and claims

A ticket is claimable only when a fresh cheap model can execute it from its own text (ADLC `coldstart`). The contract desk owns this; nobody else writes to `.adlc/tickets.json`.

Ticket contract

Field	Set by	Notes
`id`	adlc	stable; used in branch and lock names
`title`, `body`	desk	acceptance criteria each name a verification method (`spec-lint`) and cite the `PROJECT.md` line they serve
`scope` (globs)	desk	declared files; out-of-scope edits fail the gate
`deps`	desk	ticket DAG edges, each with a pinned contract
`rails`	rail author	test, contract and policy paths frozen during build
`weight`	desk	`light` (unit tests, <10 min) or `heavy` (full suite or build)
`budget`	desk	max strikes and wall-clock minutes for fleet
`tier`	desk	0 foundation, 1 shared, 2 leaf; merge order
`touches_factory`	derived	true if scope intersects `factory/`, `.adlc/`, `.github/`, `.claude/`

Field	Set by	Notes
`issue`	mirror job	GitHub Issue number, for display and comments

Gates before a ticket is claimable: `spec-lint` clean; `coldstart` gap list empty; `merge-forecast` certifies it against every other open ticket at the current width (file overlap, import-graph radius, co-change coupling); contract lint passes (scope globs resolve, `tier` set, no self-review possible). A ticket failing any of these is labeled `needs-desk` and is invisible to nodes.

Intake and the desk

The desk is a rotating weekly duty, not a role. One person per week; that week they do not run a node. The intake agent drafts tickets (scope, weight, tier, acceptance criteria with verification methods) from `PROJECT.md` and the backlog, scoring each request by risk × blast radius (tree-sitter reverse traversal from the touched symbols, the perla method). Low risk, small radius, cites the charter: the draft goes straight to spec-lint and the desk sees it in the digest. Anything else waits for the desk.

Intake rejects, and the desk does not rescue, any ticket that cannot cite the `PROJECT.md` line it serves. Refactors, dedup passes and test additions pass cold-start cleanly, and a project made only of those gets cleaner and does not move.

Desk disagreements are ADRs in `docs/adr/` through `adrs`; the desk rotation has an odd number of people.

Claim protocol

The lock is a git ref, not the Issue assignee. Creating a ref through the GitHub API fails with 422 if the ref exists, which makes it the only true compare-and-swap GitHub offers without a server. The assignee field is set after the lock for humans to read; it decides nothing.

1. Node lists Issues labeled `ready` and `w:<weight>` where weight ⩽ its capacity, oldest first, skipping any whose lock ref exists.
2. Node runs `POST /repos/{r}/git/refs` with `ref: refs/locks/issue-<n>` and `sha` = a commit on the `nodes` branch carrying `{handle, run_id, slots, claimed_at}`. On 422 or any error it moves to the next Issue. No retries on the same Issue.

3. On success: `gh issue edit <n> --add-assignee @me --add-label in-progress`, and one comment: handle, fleet run id, slot count, expected finish. This is the only comment a node writes until it has a result.
4. Heartbeat: every hour the node force-updates the lock ref to a new commit with a fresh `claimed_at`. A lock whose commit is older than 24 h is stale.
5. Release: the node deletes the ref. The janitor deletes stale refs, and any ref whose node is `stale`, and any ref beyond a node's `slots` (newest first). A stale lock with a pushed `ticket/<id>/*` branch newer than the TTL is left alone; one without a PR after 48 h is released and the branch kept for the next claimant.

No node ever deletes a lock it does not hold. No node ever edits labels other than adding `in-progress` to its own claim. Labels are set only by write access, so nothing a stranger files becomes claimable without the desk.

Rails and build gates

Rails are what the builder cannot argue with: tests, contracts and a tool policy written before any implementation exists, by an agent that will never see one, then frozen at the operating-system level for the length of the build.

Rail author

Owns specified → railed. Reads the approved spec in a fresh context; writes failing tests, type stubs and contract tests for every DAG edge; emits `.agents/permissions.json` (agent-perms: deny-first tool rules, `sandbox.writableRoots` = the ticket scope, `network.domains` = none); declares the rail paths in the ticket; runs the suite and confirms it is red for the right reasons ("not implemented", never "test broken"); hands off. Frontier-tier, one ticket per context. Must not see the builder's branch, write implementation, gate on coverage percentage, or edit rails after the freeze. Done when `hollow-test` finds no surviving mutants in the rails.

qikly's withholding check is borrowed as the enforcement: a CI step fails the rail PR if any rail file's content appears in the spec text handed to the builder, so acceptance criteria cannot leak implementation hints.

Freeze

- Ralon: the run loop launches fleet under `ralon run` with a per-ticket `agent.lock` listing the rail paths. Writes are denied by the kernel (Landlock on Linux, Seatbelt on macOS), not by a hook the agent could route around.
- Proof: CI runs `git diff --quiet origin/main -- <rail paths>` on every PR. A non-empty diff fails the gate.
- Tamper check: `tamperguard` scores the diff for new skip/xfail/ `.only` markers, deleted or loosened assertions, lowered coverage thresholds, whole-file suppressions and config loosening; exit 2 fails the gate. It runs on the node before push and again in CI.

Build gates

The node's evidence is advisory. CI reruns every deterministic gate because a laptop is not a trust boundary.

Gate	Where	Runs	Blocks merge
Build + test	CI; `factory-node` runners for `w:heavy`	project's `fleet.gate` commands	yes
Rails proof	CI	rails diff empty; tamperguard clean	yes
Scope	CI	diff ⊆ ticket scope (agent-role-contracts; notari if signing is wanted later)	yes
Bug scan	CI	`ubs` for depth; `aislop` for the Action, SARIF upload and score gate	yes
Dependencies	CI	`dependency-review-action` for license and known-vulnerable; `bomdrift` adds OSV + EPSS + KEV, typosquat and maintainer-age signals	yes

Gate	Where	Runs	Blocks merge
Flaky	CI	flaky-test-detective: JUnit history on `flaky-history`, exit codes separate flake from regression; quarantine list with expiry	real failures only
Hollow test	CI nightly on `main`	diff-scoped mutation of merged tickets (`cargo-mutants --in-diff`, Stryker incremental, mutmut; wrapped by `hollow-test`)	no; opens `needs-desk`
Gate manifest	CI	`attest` writes a signed git note: tests run, result hashes, adapter, verdicts; `attest verify` is a required check	yes

Three details that break early if left implicit:

- Dependencies. The build profile denies network, so `npm install` fails on every strike. Each node keeps a lockfile-keyed dependency cache and passes it with `--worker-deps`; CI uses the setup-action caches. Adding or upgrading a dependency is a desk ticket, never a builder decision. A Nix-based project sidesteps this entirely and is the recommendation for any new project started on the factory.
- Flaky tests. A flaky rail is a rail that lies, and fix-back will chase it forever. A failure classified as flaky does not fail the gate but opens a `needs-desk` ticket naming the test; the test stays quarantined until the desk fixes or deletes it, and quarantine entries carry an expiry date.
- Flail. `LoopGuard` tails the adapter's session log and pauses a strike that repeats the same tool call or error, or exceeds the ticket's token budget; `unasked` classifies out-of-scope edits as drift. The run loop treats either signal as a `flail` reason: push the branch, write the dead-end material to the Issue, release the lock.

Prosecution, verification, merge, acceptance, release

Prosecution is two pods, not one. Prosecutors produce findings; a separate verifier in a fresh context reproduces each one and kills the rest. Only confirmed findings spawn fix tickets.

Prosecution

Runs on the node with the node owner's quota, through the packaged `adversarial-review` CLI (`--providers` across model families, `--quorum`, `--loop` until no gating findings, exit codes derived from severity × confidence, never from a model's verdict). The run loop prefers an adapter the builder did not use when the manifest lists one, and the PR body records which adapter prosecuted so calibration can measure the difference. `adversarial-review` must be on PATH or the loop refuses to start. Prosecution depth (`prosecuteFailOn`, quorum size, loop cap) lives in `factory/policy/` under CODEOWNERS.

A sampled 1-in-5 re-prosecution runs in CI on a volunteer `factory-node` runner with a different adapter; it never blocks, it feeds calibration.

Verifier

Owns prosecuting → verified. Wraps Sauron: every finding becomes a claim with an evidence locator; mechanical checks run first (does the file, line and symbol exist; does the test the finding names fail), semantic faithfulness only for what survives. Two rules from ASImposium are hard: `REVIEWER_IS_AUTHOR` is a refusal, and dispositions are computed from reviews, never author-set. Confirmed findings become fix tickets that enter at building with the parent's rails appended; the parent waits in verified until its fix tickets are integrated. Where a finding is reproducible as a test, `edgeverdict`'s pattern applies: the verifier ships the failing test with the fix ticket.

Calibration

Monthly, on a volunteer runner: ReviewBench (219 real PRs with golden findings, local judge, per-category recall and precision) plus the Qodo planted-bug set. Recall below threshold in a category adds a lens to the prosecution charter through the normal `touches_factory` PR path. Nothing plants bugs into the project's own diffs yet; that stays a first-party extension.

Merge

GitHub's merge queue, sequential. Order is foundation-first (ADLC D2): `tier:0` before `tier:1` before `tier:2`, first-done-first within a tier, enforced by a small Action that holds a PR out of the queue while any `Depends-on:` parent is unmerged (the MergeGuard pattern, first-party because the product is SaaS). Ejection on conflict is a transition,

queued → building, with a rebase strike that does not count against the ticket's budget; the builder cherry-picks onto a fresh branch from `main`, never rebases. `autorebase` labels the conflict so the digest can show it.

Ticket completion (`adlc ticket complete --write`) happens in a post-merge CI job, never on a node, so `.adlc/tickets.json` is only written from `main`.

Preview and acceptance

Each integration branch gets a Docker Compose deployment on the heavy node with the most free slots, an address under a wildcard subdomain, and a TTL in the branch's deploy record; the janitor tears it down at TTL or on PR close. `behavior-diff` runs the project's probes against the preview and the current `main` deployment and writes a short report into the PR: routes, API responses, CLI output, DB state. `momus` is the HTTP half if the first-party probe runner proves thin.

Acceptance is the second human gate. The summoner pings one named human with the report and the address. Auto-accept when the behavior diff is empty and `touches_factory` is false; `touches_factory` tickets never auto-accept.

Release

`dsr` on the heavy node: multi-host build over SSH where the project needs more than one platform, minisign signing, SBOM, SLSA provenance, Docker canary of the installer. Trigger is accepted → released; the host list is read from node manifests with `heavy_gates: true`. When CI minutes allow, the same job runs as an Action; `dsr` exists so a throttled month does not stop releases.

Humans in the loop

Humans touch the factory in four places: the two gates, the weekly desk, and the kill switch. Everything else is designed so a human can go a week without opening the repo and still be pulled in within an hour of being needed.

Summoner

Code, not an agent: an Action triggered by hub events, posting to a Discord webhook (or email) the human already reads. Exactly two kinds of immediate message, each to one named human with the ticket id, the decision needed and a link:

- **Gate ask:** spec approval (P1) or acceptance (P6).

- **Halt notice:** kill switch set, or a node retired with open claims.

Everything else (needs-desk, quota pauses, retirements, drills, merges, ejections, flaky quarantines) goes into one daily digest, a scheduled workflow that is silent when there is nothing to report. The summoner never polls humans, never messages agents, never decides anything.

When the gate needs a recorded decision rather than a ping, `trstringer/manual-approval` is the serverless fallback: it opens an Issue assigned to the approvers and parses approve/deny comments. `awaithumans` in server mode is the upgrade if humans start colliding on approvals and typed responses are needed; it runs on a volunteer node and is not required.

Desk duty

One person per week, odd-numbered rotation. That week they approve or reject intake's drafts, resolve dead letters (needs-desk → specified or abandoned), own hotfixes and reverts, and read the digest. They do not run a node. The desk is where the week-one failure lives (ready queue at zero because writing cold-start-clean tickets is unglamorous), so intake drafts everything and the desk never starts from a blank page.

Kill switch

A file, `factory/HALT`, on `main`. Every run loop checks for it before claiming; present means sleep 30 minutes and check again. Anyone with write access can add it with a one-line PR that bypasses nothing but needs no review (the CODEOWNERS rule for `factory/HALT` requires two approvals to *remove*, zero to add, which the branch rules express as a required-reviewers rule keyed on deletion). The halt also sets the `factory:halt` label so the summoner fires a halt notice.

Removing it is the two-person rule. `slb` is the intended tool (`slb run` blocks until a peer approves, SHA-bound, logged), but its README currently states that independent-reviewer auth is not implemented and peer approvals fail closed. Until the monthly drill shows `slb` clearing a halt with two real approvers, the two-person rule is the CODEOWNERS two-approval requirement on the deletion PR and nothing else.

Drills

Once a month the janitor opens a `drill` ticket that trips the kill switch, retires a test node with an open claim, forces a merge-queue ejection, and lets a preview expire. Each must recover without a human within one janitor interval. A drill that needs a human is a

`needs-desk` ticket against the factory.

Trust and etiquette

The threat is not a malicious teammate; it is a teammate's agent reading a hostile ticket, PR or skill and leaking that teammate's credentials or wrecking their machine. Fleet's README says it plainly: model-plane reads are unbounded. So every rule below is enforced by GitHub, the kernel or a hook, never by `AGENTS.md`.

Enforced by GitHub

- `main` protected: required checks, merge queue only, no force push, no admin bypass, including for org owners.
- CODEOWNERS: `factory/**`, `.adlc/**`, `.github/**`, `.claude/**` need two human approvals. The settings that define quality are in that set: `prosecuteFailOn`, default `--max-strikes`, quorum size, branch-protection rules as code, the quarantine list, the sandbox profiles. Lowering prosecution depth is a visible PR, never a config tweak.
- Node tokens are fine-grained, repo-scoped, branch-pattern-scoped (`ticket/**`, `nodes/**`, `refs/locks/**`), 90-day expiry. Revoking a token retires a node.
- Secrets: none in the repo. CI has no model API key. Prosecution stays on nodes. Project secrets are per-node `.env`; `factory/fixtures/` holds synthetic data, and secret scanning treats fixtures as lookalikes.
- Labels are set only by write access. The claim filter is "label `ready` present", so nothing a stranger files becomes claimable without the desk. Only verifier-confirmed findings spawn fix tickets, never comments.
- Self-hosted runners on a public repo are the largest hole. `w:heavy` jobs run only on `push` events from branches matching `ticket/**` and never on `pull_request` from a fork; fork PRs get hosted runners only; "Require approval for first-time contributors" is on; runners live in a group scoped to this repo; no secrets on the runner host; ephemeral runners where the node can afford it.
- Human-authored PRs take the same gates and the same prosecution. No exemption.

Enforced on the node

- vetto sandbox, fail-closed, two profiles (Repository and nodes). Ralon path lock on the rails. `dcg` (or Vallum) hook on every shell command. ACFS safe mode only.
- The wizard refuses to install if `--dangerously-skip-permissions` is aliased.
- Every node runs the same `factory/policy/` set, pinned by `factory/VERSION`.

Skills are untrusted input. Anything under `factory/skills/` is text every node's agent will trust. skills-janitor `/janitor-security` runs in CI on every change to that directory and on each node before a skill loads; a `RISK` verdict blocks the merge. `stalebrain` and `agentslint` run weekly so a skill that describes a path or command that no longer exists is caught before an agent acts on it.

Outbound etiquette. The factory's PRs will land in other people's repos eventually (upstream fixes, dependency patches). Before any PR leaves the project, the desk's policy check reads the target's AI-contribution policy from the melissawm index and `good-oss-citizen`'s reader: a ban is a hard stop; a disclosure requirement means the `Assisted-by:` trailer and a one-line note in the PR body. `commit-check` enforces the trailer in CI. Expect `peakoss/anti-slop` on the receiving end and write PRs that would pass it: small, one concern, tests included, no boilerplate prose.

Humans erode rails before agents do. The 11pm "just this once" (hand-editing a flaky test on `main`, a temporary branch-protection exception) is the realistic threat, which is why admin bypass is off for everyone and the quality settings are code under CODEOWNERS.

House rules (`factory/RULES.md`, printed by the wizard):

- Agents sign every commit and PR with `handle`, ticket id, `Assisted-by:` and `Signed-off-by:` trailers.
- One ticket, one branch, one PR. Out-of-scope fixes become new desk requests, not drive-bys.
- Claims expire. Do not re-claim a ticket you just lost without reading its comments.
- Post your spend. The PR body carries strikes used and tokens if the adapter reports them. The number is for the group to see where quota goes, not a leaderboard.
- Bots do not open discussions, mention humans, or comment on each other's PRs. Humans talk; agents report.
- Taking your box offline for more than a day: run `/factory:release` first.
- Provider terms are each contributor's own responsibility; `CONTRIBUTING.md` says so and nothing in the factory depends on a particular reading of them.

Clock, budget, distill, observability

Four services run with nobody watching. Three are Actions on a schedule; the budget governor is a check at the top of every node loop.

Clock and janitor

An hourly Action with a `concurrency` group, first-party because nothing published combines the three jobs. It owns every time-driven transition.

- Claims: delete lock refs whose commit is older than 24 h and whose `ticket/<id>/*` branch has no commit newer than the TTL; delete locks of `stale` nodes; delete a node's newest locks beyond its `slots`. Comment on the Issue each time, in dry-run form first for a week after any janitor change (the gemini-cli pattern).
- Branches: delete `ticket/**` branches whose Issue is closed, or unclaimed for 7 days with no open PR. `crs-k/stale-branches` is the fallback age-only sweep.
- Nodes: `stale` after 24 h silent, `retired` after 7 days; token revoked, claims released, manifest archived on the `nodes` branch.
- Previews: torn down at TTL or PR close.
- Ready queue: re-run `merge-forecast` when the open set changes; recompute `ready` labels.
- Drills: the monthly `drill` ticket (Humans in the loop).

Must not: touch `main` outside the PR path, release a lock with a pushed branch newer than the TTL, run on a node, or overlap itself or the mirror job. Done when no state can be stuck for more than one janitor interval without a human noticing.

Budget governor

A check at the top of each loop, code only.

- Quota: `cux` pools the owner's accounts and reports window usage; the node sleeps when every account is at `quota_cap` (default 0.4) and resumes a strike mid-session when one rotates in. `claude-usage-limiter`'s hook is the per-project hard cap for nodes that run only Claude Code.
- Quiet hours: no claims inside the owner's declared window; a running strike finishes.
- Kill switch: `factory/HALT` present → sleep.
- Wall clock: fleet's `--wall-clock-minutes` from the ticket budget; the lock is kept and heartbeated across a pause.

A paused ticket keeps its lock. Everything the governor does is a reason code in the ledger and a line in the digest.

Distill loop

P7 is a scheduled job on a volunteer `factory-node` runner, not something a node does after its own ticket. Every output lands on `main` through the same PR path as any ticket, with `touches_factory` true, and the RRSI discipline applies: a change to `factory/` must pay for its added tokens with a measured gain on the calibration set, or it is reverted.

- Weekly `lesson-foundry` over `factory/ledger/findings.jsonl`: recurring verified findings become lint rules (PR'd with a test) or skill candidates. `CauterRule`'s replay validation is the second check: a candidate rule must also raise precision on the historical findings before promotion.
- Weekly skill rot: `stalebrain` (typed claim extraction, confidence half-lives) + `agentslint` (deterministic path/command integrity) + `skillrot` (CLI flag drift) over every `SKILL.md` and `AGENTS.md`; stale claims open a `needs-desk` ticket.
- On a model release: `model-ratchet` re-prosecutes the hot paths of `main` with the new adapter; `bug-bot --once` is the alternative cron if `model-ratchet` is unavailable. Verified findings become tickets.
- Monthly calibration (Prosecution, verification, merge).
- Gate-manifest history feeds `model-router` priors so adapters are chosen per ticket category over time, not by feel.
- `cass_memory_system` mines node transcripts locally for dead-end rules; a node reads them before each strike. Rules that recur across nodes are promoted through `lesson-foundry` as text, never as transcripts.

Observability

Two numbers first: ready-queue depth (near zero means the desk is the limit, growing means nodes are) and acceptance-queue age (humans are). A static page on GitHub Pages, generated hourly by the janitor from the `nodes` branch, the lock refs, Issue labels and `factory/ledger/`, in the `issue-dashboard` style: nodes online, claims held, queue depth by tier, acceptance age, spend by node for the week. No server, no login.

Bill of materials

Every component, its kind, and what implements it. "First-party" means the supersession pass found nothing on par and says why in the working draft; those are the places the project should expect to contribute back upstream. Tools marked † are young (under 100

stars or under six months of releases) and are wrapped behind a one-file adapter so they can be swapped.

Component	Kind	Implementation	Fallback or trial
Chassis: ticket → sandboxed run → PR	code	ADLC `fleet` composed by `ru`	agent-orchestrator or Bernstein† if fleet's adapters lag; chassis is the one open decision
Ticket graph, ready, priority	code	`.adlc/tickets.json` + Issues mirror	`tk` (git-native, `tk verify`)
Claim lock	code	git ref CAS, `refs/locks/issue-N` (action-lock pattern)	—
Intake and triage	agent	lobs-sentinel poller, re-chartered to risk × blast radius; perla's tree-sitter radius	first-party scoring
Decomposition and ambiguity	agent + code	ADLC `parallax` (fan-N readings, diff) and `coldstart`	Spec Kit `/speckit.clarify` as a cheap first pass
Ticket and role lint	code	agent-role-contracts	notari† for signed scope; branch protection for reviewer ≠ author
Conflict forecast	code	ADLC `merge-forecast`	lift co-change + import-hop code from `unasked`
Rail author	agent	ADLC P3 charter; emits agent-perms policy; qikly withholding check	TDDAgents tester role
Rails freeze	code	Ralon† (kernel path lock) + `git diff --quiet` proof + tamperguard†	tampercheck†; test-ratchet Actions

Component	Kind	Implementation	Fallback or trial
Hollow test	code	ADLC `hollow-test` over cargo-mutants `--in-diff`, Stryker incremental, mutmut	—
Builder adapter	agent	Claude Code; Codex, OpenCode, Gemini via fleet adapters	—
Sandbox	code	vetto†, two profiles, fail-closed	anthropic sandbox-runtime
Dependency cache	code	lockfile-keyed cache + `--worker-deps`; CI setup caches	Nix/devenv for new projects
Destructive command guard	code	`dcg`	Vallum† on Cursor/Gemini nodes
Flail detection	code	fleet flail reason + LoopGuard† + `unasked`†	—
Permission policy format	code	agent-permissions (`agent-perms`)	—
CI gate runner	code	GitHub Actions; `factory-node` self-hosted for heavy	Woodpecker on a volunteer node
Bug scanner	code	`ubs` + aislop	Semgrep
Dependency and license gate	code	dependency-review-action + bomdrift†	git-pkgs/actions
Flaky quarantine	code	flaky-test-detective† (history on `flaky-history`)	notmyfault
Prosecution	agent	`adversarial-review` CLI, cross-adapter, quorum, loop	llm-panel, quorum
Review calibration	code	ReviewBench + Qodo planted set, monthly	Factory review-benchmark

Component	Kind	Implementation	Fallback or trial
Verifier	agent	Sauron wrapped; ASImposium rules; edgeverdict pattern for test-shaped findings	first-party
Gate manifest and provenance	code	corvidlabs/attest† signed git notes; `aiir` for in-toto envelopes	cosign + slsa-github-generator
Merge queue	code	GitHub merge queue + `tier:` labels + first-party `Depends-on:` hold	autorebase for conflict labels
Preview environment	code	Docker Compose on a heavy node, TTL in deploy record	PullPreview if a cloud VM per PR is acceptable
Behavior diff	code	ADLC `behavior-diff` probe runner	momus† for HTTP
Summoner	code	Action + Discord webhook; two message kinds; daily digest	manual-approval Action; awaithumans server
Release	code	`dsr`	GoReleaser OSS tier
Two-person rule	code	CODEOWNERS two approvals; `slb` once the drill passes	—
Decisions	code	`adrs` with MCP	AdrMcp†
Clock and janitor	code	first-party hourly Action	crs-k/stale-branches for age-only sweeps
Kill switch	code	`factory/HALT` + CODEOWNERS removal	—
Budget governor	code	`cux` rotation + claude-usage-limiter cap + quiet hours in the loop	claude-overnight window logic
Build offload	code	`rch`	—

Component	Kind	Implementation	Fallback or trial
Node provisioning	code	`/factory:join` wizard over ACFS preflight	Nix/devenv
Distill	agent + code	`lesson-foundry` + CauterRule† replay; `model-ratchet`; `cass_memory_system`	bug-bot† for the ratchet cron
Skill rot	code	stalebrain† + agentslint† + skillrot†	skills-janitor
Memory	code	`cass` local index; `cass_memory_system` rules	deja-vu; agent-knowledge†
Observability	code	janitor-generated static page (issue-dashboard pattern)	Orca as a per-node cockpit
AI-contribution policy	code	melissawm index + good-oss-citizen† reader + commit-check trailer	—
Skill injection scan	code	skills-janitor `/janitor-security`	—

Decisions, bootstrap, failure modes

Decisions record

Every gap the cold reads, premortem, replica test and supersession pass turned up, closed. The reasoning is in the working draft; this is the record.

Decision	Resolution
Lock primitive	git ref CAS replaces the assignee as the lock; assignee is display only
Sandbox	vetto, two profiles, fail-closed; replaces hand-rolled bwrap/sandbox-exec

Decision	Resolution
Rails freeze	kernel-level (Ralon) plus diff proof plus tamper scoring; a hook alone is not a freeze
Same-vendor prosecution	prefer an adapter the builder did not use; record which prosecuted
Factory version skew	`factory/VERSION` and adapter pins; behind means no claims
Secrets and fixtures	per-node `.env`; `factory/fixtures/` synthetic; fixtures scanned as lookalikes
Bootstrap	first 48 h: one human, one node, foundation tickets only; queue opens when `merge-forecast` certifies width > 1
Human PRs	same gates, same prosecution
Desk disagreement	ADRs via `adrs`; odd-numbered rotation
Bus factor	two org owners minimum; `ru` mirrors to a second remote
Provider terms	each contributor's own; noted in `CONTRIBUTING.md`
Health signals	ready-queue depth and acceptance-queue age first
Heartbeats	`nodes` branch, never `main`
Charter	`PROJECT.md`; intake rejects tickets that cannot cite it
Concurrent Actions	`concurrency` groups on janitor, mirror, digest
Lapsed claims	next claimant starts from the lapsed branch with dead-end material appended
Transcripts	never leave the node; ledger gets hashes and reason codes
License and sign-off	chosen in `PROJECT.md` before first merge; DCO and `Assisted-by:` trailers
Drills	monthly `drill` ticket

Decision	Resolution
Self-hosting	the factory is its own first project
Two-person rule	CODEOWNERS two approvals until `slb` passes a drill with two real approvers
Quota	`cux` rotation, 0.4 cap, quiet hours default to the owner's working day
Merge ordering	`tier:` labels plus a first-party `Depends-on:` hold; SaaS queues rejected
Preview	Compose on a heavy node with TTL; per-PR cloud VMs rejected on cost
Outbound PRs	policy lookup and disclosure trailer are mandatory before any PR leaves the repo

The one open decision: the chassis. The spec assumes ADLC `fleet` composed by `ru`. The pass added two single-machine chassis that match the ticket → worktree → PR → CI-feedback loop with wider adapter support: Untrivial-ai/agent-orchestrator (mature, 20+ harnesses, Go daemon) and Bernstein (deterministic gating, 31 adapters, one month old). Neither does multi-machine over a public repo; both would sit under this spec's claim protocol and gates exactly as fleet does. The earlier candidates stand: norma, Fabro, Symphony's `SPEC.md`. The bootstrap's first factory ticket is reading Symphony `SPEC.md`, norma `docs/pdca-agent.md`, Fabro's example graphs and agent-orchestrator's adapter list, then an ADR. Whatever is chosen, the state machines, trust rules, manifest and claim protocol stay as written; only the run loop's inner command changes.

Build order

1. Clock and janitor, claim protocol, kill switch. Nothing else is safe without TTLs and a stop.
2. Join wizard and run loop with vetto, Ralon, dcg, cux.
3. CI gates and merge queue with the `Depends-on:` hold.
4. Rail author charter, prosecution CLI, verifier wrapper, summoner, digest.
5. Preview, behavior diff, release.
6. Distill, calibration, observability page.
7. Open the queue to the real project.

Implementation notes live in the ADLC repo's `docs/specs/`; read `fleet-orchestration.md` and `issue-autopilot-local.md` before step 2.

Known failure modes

Written first as a premortem, then checked against published accounts of fleets that ran. Every row is already a mechanism above; the table is here so nobody removes one without knowing what it was for.

What breaks	Why	Mechanism
Ready queue at zero, nodes idle	writing cold-start-clean tickets is unglamorous; nobody signs up	rotating desk; intake drafts, desk approves
Nodes decay from eight to two	unattended loops burn a week of quota in a Saturday; owners stop running nodes	0.4 cap, quiet hours, cux rotation
Three behaviors for one ticket	adapter CLIs drift weekly	pinned adapters in the manifest; version check
Quality drops with rails green	someone lowers prosecution depth at 11pm	quality settings under CODEOWNERS; admin bypass off
Acceptance twenty deep, channel muted	forty pings a week	two immediate message kinds; daily digest
Duplicate PRs overnight (Dostal: ~90 of 124)	two nodes take the same work	git-ref lock, merge-forecast at ready, one ticket one PR
Wrong model on 747 sessions (Dostal)	default adapter is the expensive one	adapter pinned per ticket weight in the manifest
Rate limit read as task failure (Dostal)	reason codes conflated	fleet `quota-paused` keeps the lock and resumes
Agents produce unrequested PRs that reappear after closing (Logic Inc.)	orchestrator has its own backlog	tickets only from `.adlc/tickets.json`; labels only by write access

What breaks	Why	Mechanism
456 checkout collisions in five days (Carter)	shared working tree	worktree per ticket, Ralon on rails, vetto on writes
Perf PRs with unvalidated claims (dotnet/runtime)	reviewer trusts the body	verifier reproduces or kills; evidence locators in the PR
Two reviewers carry 36% of feedback (dotnet/runtime)	review capacity is the binding constraint	auto-accept on empty behavior diff; prosecution before humans see it
The project gets cleaner and does not move	refactors pass cold-start cleanly	`PROJECT.md` citation required

What none of this fixes: a team that stops finding it fun. Every mechanism lowers the cost of participating; none creates the desire to. If ready-queue depth is zero for two weeks and the rotation has no volunteers, the factory is saying the project is done or the team has moved on, and the right response is to say so, not to add a mechanism.

Resources

Grouped by role in this spec. The working draft `Distributed Software Factory — Draft Spec v0.1` keeps the Replica test, the full supersession verdicts with the reason nothing was found per slot, and the 250-query log.

Core

- ADLC: https://github.com/voodootikigod/adlc (`ADLC.md`, `packages/fleet/README.md`, `docs/specs/fleet-orchestration.md`, `docs/specs/issue-autopilot-local.md`)
- adversarial-review CLI: https://github.com/voodootikigod/adversarial-review
- ACFS: https://github.com/Dicklesworthstone/agentic_coding_flywheel_setup (`scripts/preflight.sh`, `acfs capacity`, `acfs doctor`; safe mode only)

In the design

- Claim lock pattern: https://github.com/dnd-it/action-lock
- vetto: https://docs.rs/crate/vetto · Ralon: https://docs.rs/crate/ralon · tamperguard: https://pypi.org/project/tamperguard/ · tampercheck: https://pypi.org/project/tampercheck/

- dcg: https://github.com/Dicklesworthstone/destructive_command_guard · Vallum: https://docs.rs/crate/vallum
- agent-permissions: https://github.com/Mearman/agent-permissions · agent-role-contracts: https://github.com/suirindo/agent-role-contracts · qikly: https://pypi.org/project/qikly
- cux: https://github.com/inulute/cux · claude-usage-limiter (skill listing) · claude-overnight: https://pypi.org/project/claude-overnight/
- LoopGuard: https://pkg.go.dev/github.com/loop-eng/loopguard · unasked: https://pypi.org/project/unasked/
- ubs: https://github.com/Dicklesworthstone/ultimate_bug_scanner · aislop: https://github.com/scanaislop/aislop
- dependency-review-action: https://github.com/actions/dependency-review-action · bomdrift: https://github.com/Metbcy/bomdrift
- flaky-test-detective: https://github.com/ahmedhashmu/flaky-test-detective · notmyfault: https://github.com/tashikomaaa/notmyfault
- cargo-mutants `--in-diff`: https://mutants.rs/in-diff.html · Stryker incremental, mutmut, PIT by name
- Sauron: https://github.com/rsanayei/Sauron · ASImposium: https://asimposium.org/ · edgeverdict: https://github.com/anp0429/edgeverdict
- ReviewBench: https://github.com/review-bench/ReviewBench · Qodo PR-Review-Bench (Hugging Face dataset)
- attest: https://github.com/corvidlabs/attest · aiir: https://github.com/invariant-systems-ai/aiir
- autorebase: https://github.com/movelab-studio/autorebase · MergeGuard (pattern only): https://docs.mergeguard.dev/features/pr-dependencies
- AEnvironment (preview primitive): https://github.com/inclusionAI/AEnvironment · momus: https://docs.rs/crate/momus-diff
- manual-approval: https://github.com/marketplace/actions/manual-workflow-approval · awaithumans: https://github.com/awaithumans/awaithumans-human-in-the-loop-ai-agents
- dsr: https://github.com/Dicklesworthstone/dsr · rch: https://github.com/Dicklesworthstone/remote_compilation_helper · ru: https://github.com/Dicklesworthstone/repo_updater · slb: https://github.com/Dicklesworthstone/slb
- adrs: https://github.com/joshrotenberg/adrs

- lesson-foundry, model-ratchet, hollow-test, merge-forecast, parallax, coldstart, behavior-diff, review-calibration: ADLC toolkit
- CauterRule: https://pypi.org/project/cauterule/ · stalebrain: https://pypi.org/project/stalebrain/ · agentslint: https://pypi.org/project/agentslint/ · skillrot: https://github.com/mishanefedov/skillrot · skills-janitor: https://github.com/khendzel/skills-janitor
- cass, cass_memory_system, meta_skill: https://github.com/Dicklesworthstone/coding_agent_session_search · https://github.com/Dicklesworthstone/cass_memory_system · https://github.com/Dicklesworthstone/meta_skill · deja-vu: https://github.com/vshulcz/deja-vu
- issue-dashboard: https://github.com/ethomson/issue-dashboard · crs-k/stale-branches: https://github.com/crs-k/stale-branches · gemini-cli unassign workflow: https://raw.githubusercontent.com/google-gemini/gemini-cli/main/.github/workflows/unassign-inactive-assignees.yml
- lobs-sentinel: https://github.com/lobs-ai/lobs-sentinel · perla: https://pypi.org/project/perla/
- AI-contribution policies: https://github.com/melissawm/open-source-ai-contribution-policies · good-oss-citizen: https://github.com/tesslio/good-oss-citizen · commit-check: https://github.com/commit-check/commit-check · anti-slop: peakoss/anti-slop · chaoss/ai-detection-action · good-egg: https://github.com/2ndSetAI/good-egg
- Runner hardening: https://github.com/dduzgun-security/github-self-hosted-runners · https://securitylab.github.com/resources/github-actions-preventing-pwn-requests/

Trial candidates

- Chassis: Untrivial-ai/agent-orchestrator · Bernstein: https://bernstein.readthedocs.io · norma: https://github.com/normahq/norma · Fabro: https://github.com/fabro-sh/fabro · Symphony: https://github.com/openai/symphony
- Ticket graph: `tk`: https://github.com/EnderRealm/ticket · vissue: https://github.com/HaoZeke/vissue · beads_rust, beads_viewer (Dicklesworthstone)
- Scope lint: notari: https://pypi.org/project/notari/ · agentdiff: https://pypi.org/project/agentdiff-cli/
- Sandbox: anthropic-experimental/sandbox-runtime
- Ratchet cron: bug-bot: https://github.com/BrokkAi/bug-bot
- ADR: AdrMcp: https://github.com/Atypical-Consulting/AdrMcp
- Memory: agent-knowledge: https://github.com/keshrath/agent-knowledge

- Policy and attestation: git-pkgs/actions: https://github.com/git-pkgs/actions

Reading

- Symphony `SPEC.md`; LayerFS: https://github.com/Ephemeral-AI-Lab/layerfs; Orca: https://github.com/stablyai/orca; RRSI: https://github.com/google-research/rrsi · https://arxiv.org/abs/2609.24972; AgInt: https://arxiv.org/abs/2610.23996; Weng, Harness Engineering: https://lilianweng.github.io/posts/2026-07-04-harness/
- Papers behind first-party slots: SpecFix https://arxiv.org/abs/2505.07270 · Co-Coder partitioning https://arxiv.org/abs/2606.00953 · SKILLGUARD https://arxiv.org/pdf/2605.10990 · TestSuiteBench https://arxiv.org/abs/2610.13065 · Hora & Robbes on AI policies https://arxiv.org/abs/2605.16706 · "Where Do AI Coding Agents Fail?" https://arxiv.org/abs/2601.15195
- Postmortems: Answer.AI https://www.answer.ai/posts/2025-01-08-devin · Dostal https://mdostal.com/blog/35-agent-ai-coding-swarm · Logic Inc. https://bits.logic.inc/p/losing-control-of-our-coding-agents · Toub https://devblogs.microsoft.com/dotnet/ten-months-with-cca-in-dotnet-runtime/ · Carter https://brianbcarter.substack.com/p/claude-code-is-a-horrible-scientist · Copes https://flaviocopes.com/agentic-ai-lessons/ · Willison https://simonwillison.net/2026/feb/12/an-ai-agent-published-a-hit-piece-on-me · RedMonk policy landscape https://redmonk.com/kholterhoff/2026/02/26/generative-ai-policy-landscape-in-open-source/
- Platform: GitHub Actions billing https://docs.github.com/billing/managing-billing-for-your-products/managing-billing-for-github-actions · Copilot coding agent https://docs.github.com/en/copilot/concepts/agents/coding-agent/about-coding-agent

Set aside

agent-taskbay (HITL console), agent-authz (MCP gateway), AEnvironment's Kubernetes plane, mcp_agent_mail (file leases as fallback), ntm and frankenterm (per-node cockpit), frankengit (exit if GitHub becomes the constraint), eidetic_engine_cli, Foremerge (intent-declaration overlap), HomeFleet (LAN delegation), PullPreview, Spec Kit, Opik, smartcontractkit/quarantine, pamelafox/issue-triager-agent, nibzard/awesome-agentic-patterns, Plazmaz/leaky-repo.

Rejected, with reason

SaaS core: Retasc, MergeGuard, Curtail, pliuz, LoopPause, Deconflict, PackVault, flaky-quarantine, repo-ranger. Dormant or server-bound: bors-ng, sudo_pair, pyadr, log4brains, apb7/unassign, Summoner-Network. Withdrawn: vyuha. Single-commit: diffgate,

CodebaseCSI. Non-OSS licence: rai-lint. Fails open: Leash (hoop.dev). Name coincidence: AgentRail, brege/agent-janitor, gootf/HALT.